

Apuntes Semana 5 Clase 2 Inteligencia Artificial

1st Roberto José Garita Mata
Escuela de Ingeniería en Computación
Instituto Tecnológico de Costa Rica
Cartago, Costa Rica
robertogarita99@estudiantec.cr

Profesor: Steven Andrey Pacheco Portuquez
Curso: IC6200 Inteligencia Artificial
Fecha: Martes 19 de Marzo

Abstract—En este presente documento se van a presentar los apuntes de la clase del 17 de Marzo, correspondiente a la semana 5 del curso de Inteligencia Artificial, con el profesor Steven Pacheco. Se va a comenzar como es habitual con un pequeño repaso de la clase anterior (Martes 17 de Marzo) sobre el tema del Sesgo y Varianza y seguidamente se continua propiamente ya con los apuntes del día correspondiente, Martes 19 de Marzo, sobre el tema de la Regresión Logística.

I. REPASO DE LA CLASE ANTERIOR (MARTES 17 DE MARZO)(SESGO Y VARIANZA)

A. Dataset

El conjunto de datos se divide generalmente en dos partes: *training* y *test*. Usualmente, se emplea un 80% para entrenar el modelo y un 20% para evaluarlo con datos no vistos.

B. Training set

El conjunto de entrenamiento se usa para ajustar el modelo y aprender patrones a partir de los datos. Debe ser representativo para que el modelo pueda generalizar, de lo contrario, puede ocurrir *overfitting*.

C. Testing Set

El conjunto de prueba se usa para evaluar el modelo con datos nuevos, es decir, datos que no vio durante el entrenamiento. Sirve para medir su rendimiento real y verificar si puede generalizar correctamente.

D. Caso Overfitting

El *overfitting* ocurre cuando el modelo obtiene muy buenos resultados en entrenamiento, pero falla al evaluarse con datos nuevos. Esto indica que aprendió demasiado los detalles del conjunto de entrenamiento y no logra generalizar adecuadamente.

E. Validation Set

Es el subconjunto de datos que se usa para medir qué tan bien generaliza el modelo durante el entrenamiento. Sirve para tomar decisiones y ajustar hiperparámetros.

1) **Random Sampling**: Divide el dataset de forma aleatoria. Funciona mejor cuando las clases están balanceadas, pero en datos desbalanceados puede dejar clases poco representadas.

2) **Stratified Sampling**: Se usa en datos desbalanceados. Mantiene la proporción de cada clase en cada subconjunto, logrando una evaluación más confiable.

3) **K-Fold / Cross-Validation**: Divide los datos en K partes, entrena con K-1 y valida con la restante, repitiendo el proceso varias veces. Es útil cuando hay pocos datos.

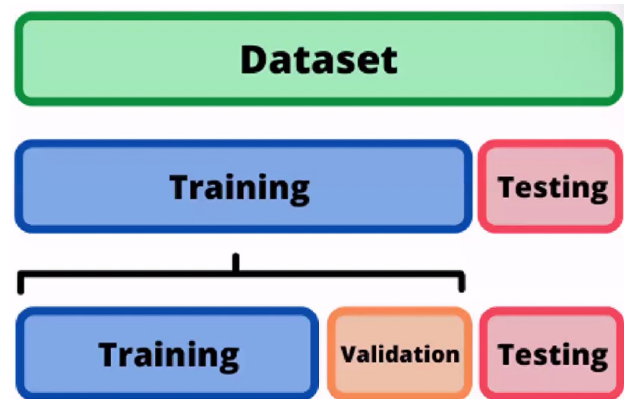


Fig. 1. División del dataset

F. Posibles escenarios

1) Bajo error en training y en validation:

- Escenario ideal
- Modelo evita el ruido existente en los datos
- Se puede generalizar correctamente

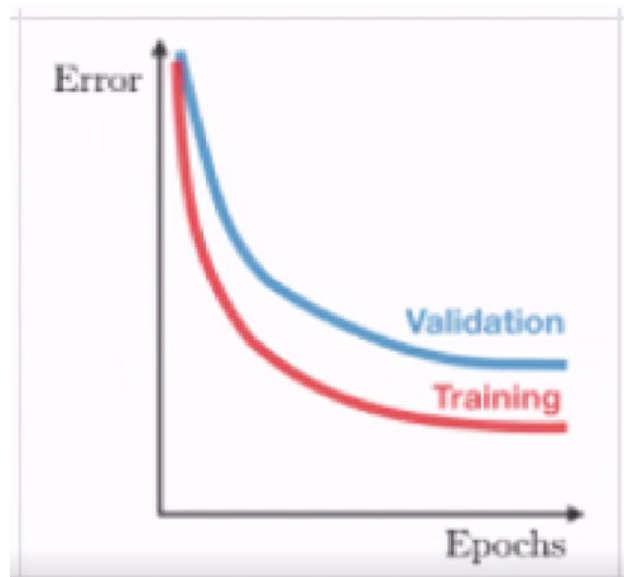


Fig. 2. Escenario ideal entre validación y entrenamiento

2) **Bajo error en training pero alto en validation:**

- Tiene overfitting
- No es capaz de generalizar
- Tiene alta varianza



Fig. 3. Caso de overfitting

3) **Alto error en training y en validation:**

- Tiene overfitting
- El modelo no está aprendiendo nada de los datos
- Modelo muy simple
- Contiene alto sesgo



Fig. 4. Pésimo escenario, caso de underfitting

1) **Alto Bias:**

- Comete muchos errores en el training (underfitting)
- El modelo asume mucho del training set
- No usa todos los features del modelo
- El modelo es simple

2) **¿Como evitarlo?:**

- Utilizando un modelo más complejo
- Los features del training set no son los adecuados para el problema (No tienen la capacidad para hacer predicciones)

3) **Alta Varianza:**

- El modelo se adapta mucho a los datos de entrenamiento (overfitting)
- No es capaz de generalizar o sea es sensible a las variaciones de los datos
- Pasa a menudo con datos con muchas dimensiones y pocos ejemplos

4) **¿Como evitarlo?:**

- Intentar con un modelo más simple
- Reducir la dimensionalidad
- Obtener más ejemplos para el training set
- Aplicar técnicas de regularización o sea hacer el modelo más simple

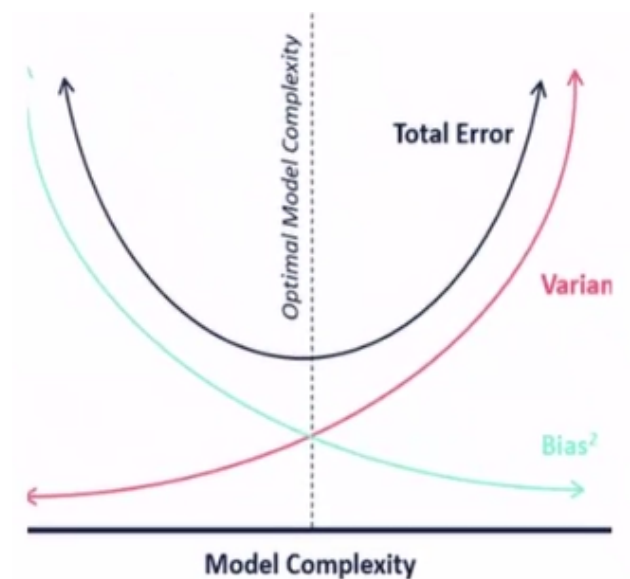


Fig. 5. Comparación de la complejidad del modelo

G. *Bias-Variance tradeoff*

Se necesita un modelo que tenga baja varianza y bajo sesgo.

II. REGRESIÓN LOGÍSTICA

Regresión Logística != Regresión Lineal

Una regresión lineal son features que tienen relaciones lineales con una variable objetivo, lo que hace es una combinación lineal entre los pesos y los features para determinar cual de ellos es más importante y a partir de esto hacer una predicción de valores reales, al final es una predicción de valores en espacios donde no tengo muestras.

La regresión logística es un algoritmo de clasificación binaria que permite predecir basado en la probabilidad de que un evento ocurra o no. Su salida representa una de dos clases posibles, por lo que se utiliza para problemas donde se desea decidir entre dos categorías específicas.

Ambos tienen nombres similares pero hacen cosas totalmente diferentes a pesar de esto.

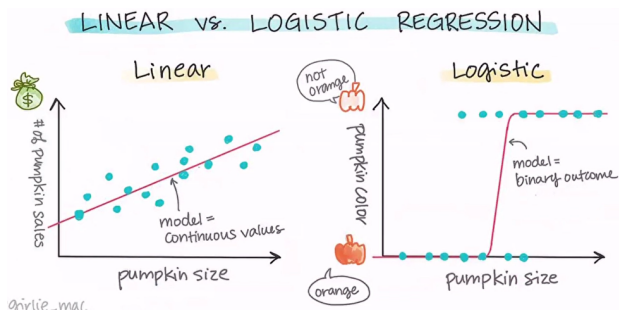


Fig. 6. Regresión Lineal vs Regresión Silogística

- Es una clasificación binaria (0s o 1s)
- Predice la probabilidad de ocurrencia de un evento binario
- Lo hace mediante la distribución de Bernoulli:

$$P(X = k) = p^k \cdot (1 - p)^{1-k}$$

- Donde k es el valor que puede tomar (0 o 1) y p la probabilidad de que ocurra el evento
- Debemos aplicar este modelo pero con machine learning para que se pueda aprender solo de los datos. ¿Cómo logramos esto? Con la función Sigmoid



Fig. 7. Meme

A. Sigmoid (Standar Logistic Function)

- Tiene un comportamiento no lineal
- Acotada entre 0 y 1
- Su función es:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

- x puede ser cualquier número, incluso el resultado de otra función.
- ¿Cómo se puede combinar con regresión lineal?
- Para combinarlo con regresión lineal a la salida de esta se le coloca una función Sigmoid esto vuelve esa regresión lineal se vuelve un problema de clasificación y con esto estamos logrando mantener la combinación lineal entre pesos y features pero se aplica ahora para problemas de clasificación.

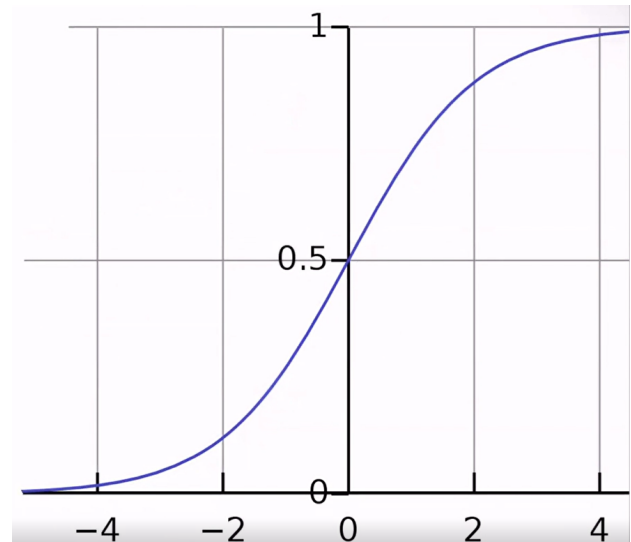


Fig. 8. Función Sigmoid

B. Linealidad (explicación del punto anterior)

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Resultado no lineal

$$f_{w,b}(x) = wx + b$$

Resultado lineal

$$\sigma(f_{w,b}(x)) = \frac{1}{1 + e^{-(f_{w,b}(x))}}$$

- ¿Por qué queremos hacerlo así?
- Calcular una función lineal es sumamente simple (computacionalmente)
- Es un método simple para mantener la relación entre las variables y los pesos
- Obtener un comportamiento no lineal con una función sencilla como lo es la Sigmoid
- Permite modelar problemas con una complejidad mayor

C. Clasificador

- Si queremos hacer un clasificador, hay que definir un umbral, que por lo general es 0.5

- Si $y \geq 0.5 = 1$ pertenece a la clase 1
- Si $y < 0.5 = 0$ pertenece a la clase 0
- De igual manera el umbral puede cambiar, dependiendo de la situación

D. ¿Cómo se ve mi modelo de Regresión Logística?

- Su función es:

$$\sigma(f_{w,b}(x)) = \frac{1}{1 + e^{-(f_{w,b}(x))}}$$

- Su resultado es no lineal
- Acotación entre 0 y 1
- La relación de los features y pesos se da por la regresión lineal
- Probabilidad de que un evento ocurra

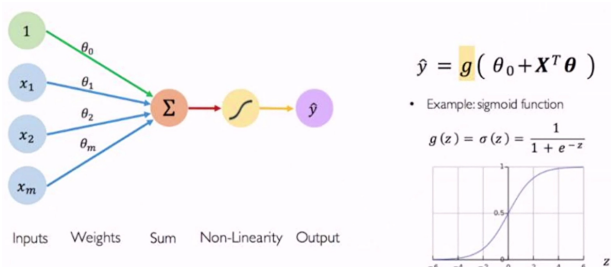


Fig. 9. Gráficos más comunes encontrados en literaturas sobre Regresión Logística, arquitectura a nivel gráfico

- Inputs: vector que define las características del sample, la muestra
- El 1 que vemos es el Bias
- Weights: pesos
- Sum: sumatoria de la relación entre las características y su peso asociado
- Non-Linearity: es una capa extra, es función no lineal, es la función Sigmoid
- Output: es una salida, una probabilidad, que a partir de esto se toma una decisión

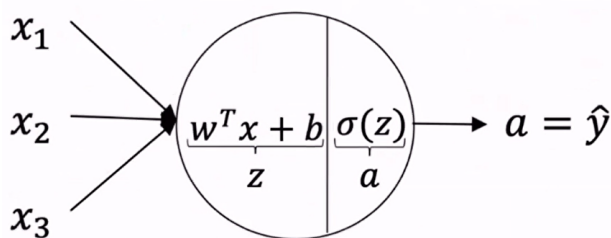


Fig. 10. Función de Activación(Función Sigmoid)(función no lineal)

En el fondo este gráfico, lo que representa que es si deja pasar información o no deja pasar información y en un caso específico transformarla. Las funciones de activación son funciones que tienen un comportamiento no lineal, existen más funciones de activación pero las veremos más adelante.

E. Optimización

- Para optimizar nuestro modelo se necesita optimizar los pesos w y b de la regresión logística
- Dado por la función: $f_{w,b}(x) = \frac{1}{1 + e^{-(f_{w,b}(x))}}$
- ¿Qué necesitamos para actualizar los pesos?
- 1) Encontrar una función de pérdida
- 2) Encontrar las derivadas parciales con respecto a w y b , para utilizar el descenso del gradiente
- 3) Se actualizan los parámetros w y b una vez calculadas las derivadas con respecto a la función de pérdida

F. Derivada Sigmoid

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

$$\sigma'(x) = \frac{1' \cdot (1 + e^{-x}) - (1 \cdot (1 + e^{-x})')}{(1 + e^{-x})^2}$$

$$\sigma'(x) = \frac{0 - 1 \cdot (1' + (e^{-x})')}{(1 + e^{-x})^2}$$

$$\sigma'(x) = \frac{-(0 - (-e^{-x}))}{(1 + e^{-x})^2}$$

$$\sigma'(x) = \frac{e^{-x}}{(1 + e^{-x})^2}$$

Podemos observar que es medio fea de utilizar entonces vamos a usar un truco para que quede un poco mejor.

$$\sigma'(x) = \frac{e^{-x}}{(1 + e^{-x})^2}$$

$$\sigma'(x) = \frac{e^{-x} + 1 - 1}{(1 + e^{-x})^2}$$

$$\sigma'(x) = \frac{e^{-x} + 1}{(1 + e^{-x})^2} - \frac{1}{(1 + e^{-x})^2}$$

$$\sigma'(x) = \frac{1}{(1 + e^{-x})} - \frac{1}{(1 + e^{-x})^2}$$

$$\sigma'(x) = \frac{1}{(1 + e^{-x})} \cdot \left(1 - \frac{1}{(1 + e^{-x})}\right) \quad \text{Factor común}$$

$$\sigma'(x) = \sigma(x) \cdot (1 - \sigma(x))$$

G. Verosimilitud vs MSE

- NO usamos MSE
- Necesitamos una función de costo relacionada a probabilidades
- Verosimilitud es la probabilidad condicional que se obtiene de observar mi conjunto de entrenamiento, osea mis datos dados los parámetros de un modelo:

$$L(\theta|X) = P(X|\theta)$$

- Probabilidad de observar todos los datos:

$$L(\theta|X) = P(x_1, x_2, \dots, x_n|\theta) = P(x_1|\theta) \cdot P(x_2|\theta) \cdots P(x_n|\theta)$$

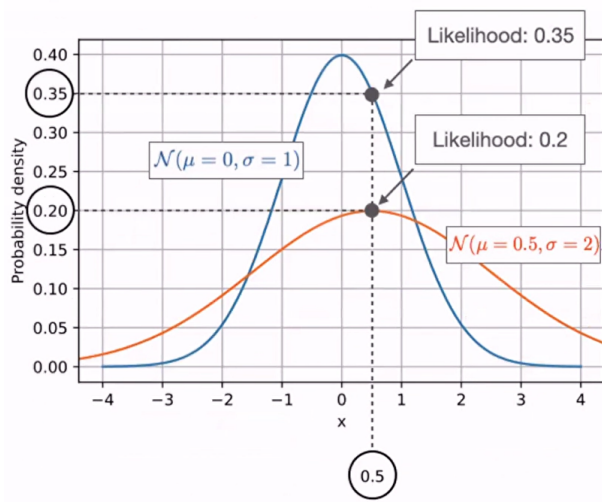


Fig. 11. Afectación de la varianza de los datos

EJEMPLOS

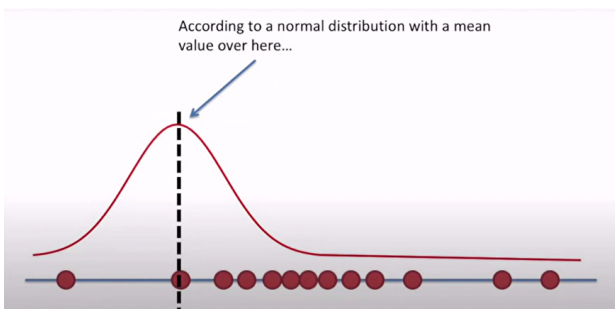


Fig. 12. Verosimilitud baja

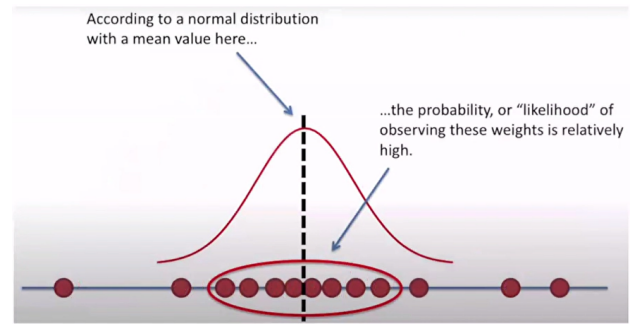


Fig. 13. Verosimilitud alta

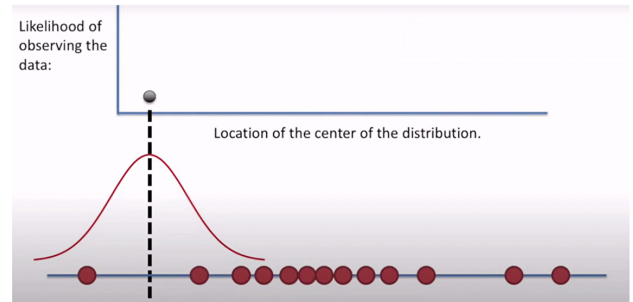


Fig. 14. Verosimilitud 1

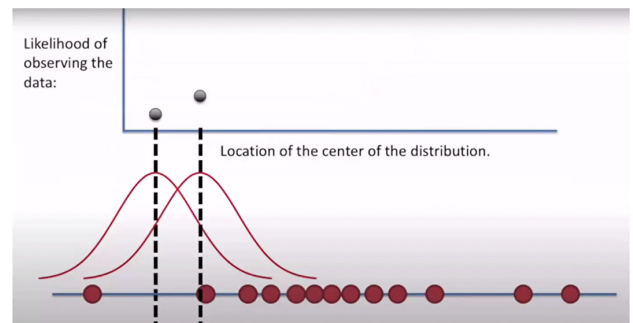


Fig. 15. Verosimilitud 2

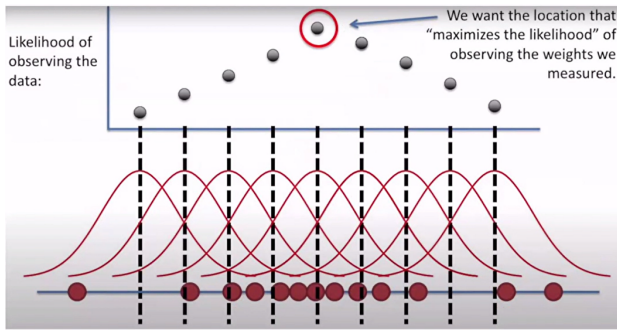


Fig. 16. Máxima Verosimilitud

Conforme yo cambie los parámetros de mi modelo lo que hago es que mi función avance y eventualmente voy a descubrir que habrán un conjunto de parámetros específicos que hacen que mi distribución sea la óptima para los datos que yo tengo y por lo tanto va a dar un resultado del Likelihood alto. En el fondo lo que estamos haciendo es maximizando la verosimilitud.

H. MSE vs Maximize Likelihood

- En regresión lineal minimizamos el MSE
- En su lugar, vamos a maximizar la verosimilitud
- Optimizamos los parámetros w y b , para obtener la máxima verosimilitud